

EngineeringOS — الخطة التنفيذية الشاملة لوكيل الذكاء الاصطناعي

(1) الهدف النهائي

EngineeringOS ليس مجرد لوحة تحكم أو مجموعة ميزات، بل طبقة معرفة هندسية تُبنى فوقها أدوات التطوير الذكية. الهدف النهائي هو أن يصبح النظام قادرًا على:

1. فهم المشروع من داخله لا من واجهته.
2. استخراج الحقيقة البنيوية من الملفات والكود والعلاقات والاعتمادات.
3. تحويل هذه الحقيقة إلى مهام، قواعد، مخططات، مقاييس، وسير عمل قابل للتنفيذ.
4. ضمان أن كل قرار تنفيذي قابل للتتبع، والتدقيق، وإعادة البناء.
5. تقديم طبقة معرفة موثوقة يمكن أن تبني فوقها أدوات AI مستقبلية.

(2) الحالة الراهنة التي يجب أن ينطلق منها الوكيل

المستودع الحالي يملك أساسًا معماريًا واضحًا ومقسّمًا إلى طبقات:

- طبقة العقد: OpenAPI كمصدر حقيقة، مع توليد Zod و React Query.
- طبقة البيانات: Drizzle + PostgreSQL.
- طبقة التحليل: Scanner مستقل لاستخراج الملفات والعلاقات والرسوم البيانية.
- طبقة التنفيذ: API server لتنسيق الفحص والمهام والقواعد والسير.
- طبقة العرض: Dashboard React.
- طبقة المختبر: mockup sandbox للتجريب غير الإنتاجي.

كما أن المشروع بالفعل يملك:

- سجل حقيقة مفصل file-by-file.
- خطة مرحلية مع ترتيب صحيح للعمل من الداخل إلى الخارج.
- أساسًا جيدًا في العقد، البيانات، والفحص.
- بداية قوية في الرسوم البيانية، الأحداث، وسير العمل.
- حاجة واضحة لتعميق طبقة المعرفة نفسها، لا مجرد عرضها.

(3) القواعد الحاكمة التي لا يجوز كسرها

على الوكيل أن يلتزم بهذه القواعد في كل خطوة:

1. العقد أولاً

أي API جديد أو تعديل في API يبدأ من `lib/api-spec/openapi.yaml`، ثم codegen، ثم التطبيق.

2. العمل من الداخل إلى الخارج
لا يبدأ من الواجهة، بل من البيانات، ثم التنفيذ، ثم التحليل، ثم المعرفة، ثم العرض.
3. كل تغيير يجب أن يكون قابلاً للتحقق
لا يُعتبر أي تعديل منجزاً إلا إذا مرّ:
 4. typecheck
 5. build
 6. tests
 7. codegen drift check عند الحاجة
 8. لا تغييرات غير موثقة
أي تعديل جوهري يجب أن ينعكس في:
 9. docs/fact-record.md
 10. docs/completion-plan.md
 11. و/أو replit.md إذا تغيّر التشغيل أو المسارات
 12. الاعتماد على الأدلة لا على الافتراضات
لا يفترض الوكيل وجود ميزة مكتملة فقط لأنها مذكورة في الوصف.
المرجع هو الكود الفعلي.
 13. الذرة والاتساق أهم من السرعة
كل عملية import / discover / scan / workflow / task execution يجب أن تكون:
 14. atomic حيث يلزم
 15. transactional حيث يلزم
 16. idempotent قدر الإمكان
 17. قابلة لإعادة المحاولة بأمان

(4) استراتيجية التنفيذ العامة

الوكيل يعمل وفق حلقة ثابتة:

1. فحص الحقيقة الحالية
قراءة الملفات المعنية، مقارنة الوثائق بالكود، تحديد الفجوات.
2. اختبار أصغر وحدة ذات أثر حقيقي
لا يوسع النطاق بلا داعٍ، كل مهمة يجب أن تغلق فجوة واحدة واضحة.

3. تنفيذ التغيير من الطبقة الأدنى المؤثرة
مثال: إذا كانت المشكلة في البيانات، لا تُعالج في الواجهة أولاً.

4. التحقق الفوري
تشغيل الاختبارات، typecheck، build، codegen عند الحاجة.

5. تحديث السجل والوثائق
أي حقيقة جديدة يجب أن تُسجل.

6. التوقف عند الفشل
لا يتجاوز الوكيل فشلاً غير مفهوم. يجب عزل السبب ثم إصلاحه قبل المتابعة.

5) الخطة التنفيذية المرحلية

المرحلة 0 — تثبيت الحقيقة المرجعية

الهدف: جعل الوثائق تعكس الوضع الفعلي بدقة.

المهام:

- مراجعة docs/fact-record.md مقابل شجرة الملفات الفعلية.
- مراجعة docs/completion-plan.md مقابل ما نُفذ فعلاً.
- تحديث أي أرقام أو ملفات مضافة أو محذوفة.
- التأكد من أن أي ذاكرة تشغيلية داخل agents/memory/ لا تتعارض مع الواقع الحالي.

مخرجات النجاح:

- لا توجد ملفات مفقودة أو غير موثقة.
- لا توجد ادعاءات معمارية غير مدعومة بالكود.
- كل خطة لاحقة مبنية على واقع صحيح.

المرحلة 1 — تثبيت القاعدة التقنية

الهدف: ضمان أن البنية الأساسية آمنة، متماسكة، وسهلة التوسع.

المهام:

- تعزيز قيود قاعدة البيانات حيث تكون هناك أخطار فعلية.
- توحيد الإعدادات (config) في مسار واحد.
- تثبيت سلوك الاستجابة في الـ API.
- مراجعة naming conventions ومنع أي تباين غير مبرر.
- تثبيت تدفقات الأخطاء في المسارات الحرجة.

مناطق التركيز:

- lib/db/src/schema/*
- artifacts/api-server/src/config.ts
- artifacts/api-server/src/routes/*
- artifacts/api-server/src/lib/*

معايير الإنجاز:

- لا توجد حالات سباق في claim/retry/import.
- لا توجد استجابات متفرقة أو غير متوقعة.
- لا ينهار الاختبار بسبب config غير متناسق.
- لا توجد علاقة data corruption محتملة غير مغطاة.

المرحلة 2 — استكمال محرك سير العمل والقواعد

الهدف: جعل workflow engine و rule engine حقيقيين لا شكليين.

المهام:

- تثبيت state machine للسير.
- ضمان retry / rollback / fail / resume بشكل ذري.
- ربط rule violations بتاريخ واضح.
- حفظ trace كامل لكل transition.
- ربط التنفيذ بالأحداث بالـ audit/event logs.

المخرجات المتوقعة:

- كل workflow execution له تاريخ واضح.
- كل retry أو rollback له سبب وحدث.
- لا توجد transitions غير صالحة.
- لا يوجد execution "معلق منطقياً" بدون أثر.

معايير النجاح:

- transitions ممنوعة إلا وفق القواعد.
- لا توجد كتابة جزئية عند الفشل.
- كل change في workflow يظهر في audit/event trail.

المرحلة 3 — تعميق scanner و discovery

الهدف: تحويل scanner من أداة فحص إلى أداة فهم بنيوي للمشروع.

المهام:

- تعميق استخراج AST والاعتماديات الفعلية.
- تحسين فهم الأنماط البرمجية الأوسع.
- دعم علاقات أكثر دقة بين الملفات والكيانات.
- تثبيت discovery على rootPath الصحيح دون fallback مضلل.
- تحسين معالجة المشاريع المستوردة والمرصودة.

ما يجب أن ينتج:

- scan reports أكثر ثراء.
- graph relationships أدق.
- اكتشافات أقل ضجيجًا وأكثر موثوقية.
- نتائج قابلة للبناء عليها في طبقة المعرفة.

معايير النجاح:

- لا يُفحص مسار خاطئ.
- لا تختلط الملفات المحلية بمصادر أخرى.
- لا تُسجل علاقات ضعيفة على أنها حقيقة قوية.

المرحلة 4 — تحويل الرسم البياني إلى طبقة معرفة

الهدف: أن يصبح graph layer بنية معرفة قابلة للاستعلام والتحليل، وليس مجرد مخزن روابط.

المهام:

- تعريف الكيانات بشكل أوضح.
- تثبيت أنواع العلاقات ومعانيها.
- ربط provenance بكل عنصر مهم.
- توفير استعلامات الجوار والتجميع والاشتقاق.
- دعم تمثيل المعرفة عبر الزمن، لا فقط اللحظة الحالية.

المخرجات:

- يمكن فهم المشروع عبر graph semantics.
- يمكن تتبع "لماذا هذه العلاقة موجودة؟"
- يمكن استنتاج صورة أعلى مستوى من العلاقات الخام.

معايير النجاح:

- أي علاقة مهمة لها مصدر واضح.
- graph queries تعكس الواقع الفعلي.
- المعرفة قابلة للتدقيق لا مجرد العرض.

المرحلة 5 — تعميق طبقة التنفيذ

الهدف: أن يصبح الـ API server منسقًا موثوقًا بين المعرفة والتنفيذ.

المهام:

- تحسين orchestration بين scan/jobs/tasks/workflows.
- ضمان إدارة الحالة بشكل صحيح.
- توسيع audit coverage.
- توحيد logging و correlation.
- تقليل أي منطق متكرر أو مبعثر.

معايير النجاح:

- لا توجد عمليات خلفية غير منضبطة.
- كل job يمكن تتبعه من البداية للنهاية.
- يمكن استئناف/إعادة التحقق من الحالات الحرجة.

المرحلة 6 — تعميق واجهة التحكم

الهدف: أن تعكس الواجهة الحقيقة الداخلية بوضوح تشغيلي.

المهام:

- عرض history للمهام والسير.
- عرض graph neighborhoods بشكل مفيد.
- إظهار trends للمقاييس.
- عرض logs و states و failures بطريقة تشغيلية.
- تحسين empty/error states لتخدم التشخيص.

المبدأ: الواجهة لا تختصر الحقيقة، بل تكشفها.

معايير النجاح:

- المستخدم يرى ما يحدث فعليًا.
- لا توجد صفحة "جميلة" لكنها مخادعة أو مبسطة أكثر من اللازم.
- كل شاشة تساعد على اتخاذ قرار أو تشخيص.

المرحلة 7 — توسيع الاختبارات والملاحظة

الهدف: حماية المنصة من الانحدار.

المهام:

- اختبارات المسارات الحرجة والفشل.

- اختبارات التزامن والسباقات.
- اختبارات drift بين العقد والتوليد.
- اختبارات discovery/import/scan/workflow.
- تحسين observability للأخطاء والسلاسل التنفيذية.

معايير النجاح:

- أي تغيير حساس لا يمر دون اختبار.
- لا توجد فجوات صامتة بين العقد والتطبيق.
- مسارات الفشل مغطاة، لا فقط المسارات السعيدة.

المرحلة 8 — إكمال قابلية التوسعة والملحقات

الهدف: جعل EngineeringOS منصة يمكن البناء فوقها، لا فقط استخدامها داخليًا.

المهام:

- تثبيت plugin contract.
- إنشاء registry داخلي منظم.
- ربط plugins بالنقاط المناسبة في النظام.
- تقديم مثال عملي لملحق واحد على الأقل.
- الحفاظ على الأمان والعزل في الامتداد.

معايير النجاح:

- الإضافة لا تكسر core.
- الامتداد منضبط بالعقد.
- لا يوجد plugin framework نظري فقط.

المرحلة 9 — التوثيق النهائي والتسليم

الهدف: تحويل الإنجاز التقني إلى حزمة قابلة للفهم والإرسال والتسليم.

المهام:

- إبقاء `fact-record` و `completion-plan` محدثين.
- تلخيص ما اكتمل وما تبقى.
- ربط التشغيل الفعلي بالوثائق.
- كتابة handoff واضح لأي مطور أو وكيل لاحق.

معايير النجاح:

- لا توجد فجوة بين "ما يقال" و "ما يعمل".
- يمكن لأي طرف جديد فهم المشروع من الوثائق والتشغيل.
- الخطة المستقبلية ناتجة من الواقع لا من التمني.

6) ترتيب الأولويات العملي للوكيل

الترتيب الصحيح للتنفيذ هو:

1. الحقيقة والوثائق
2. القاعدة التقنية والقيود
3. workflow/rules
4. scanner/discovery
5. graph/knowledge layer
6. API orchestration
7. UI operational depth
8. tests/observability
9. plugin extensibility
10. documentation/handoff

أي انحراف عن هذا الترتيب يجب أن يكون له سبب واضح جدًا، لأن أي تقدم مبكر في الواجهة أو الامتدادات قبل تثبيت القاعدة سيؤدي إلى طبقة لامعة فوق أساس غير مستقر.

7) تعريف "المنجز" بالنسبة لوكيل الذكاء الاصطناعي

أي مهمة لا تُعتبر منجزة إلا إذا تحققت الشروط التالية:

- الكود يعمل.
 - الاختبارات تمر.
 - typecheck يمر.
 - build يمر.
 - لا يوجد drift في العقد المولدة.
 - الأثر موثق.
 - لا يوجد سلوك جانبي غير مقصود.
 - لا توجد حالة فشل حرجة غير مغطاة.
-

8) صيغة التشغيل اليومية للوكيل

عند العمل داخل EngineeringOS، يتبع الوكيل هذا النمط اليومي:

1. يقرأ الحالة الحالية.
2. يحدد مهمة واحدة ذات أثر.
3. يغير أقل مجموعة ملفات ممكنة.
4. يشغل التحقق.
5. يصلح الفشل فورًا.
6. يوثق النتيجة.
7. ينتقل للمهمة التالية فقط بعد الإغلاق الكامل.

(9) النتيجة المتوقعة عند اكتمال الخطة

عند اكتمال هذه الخطة، سيصبح EngineeringOS:

- منصة هندسية قائمة على الحقيقة الداخلية للمشروع.
- طبقة معرفة فوق البيانات والكود والعلاقات.
- نظامًا يربط discovery بال graph بال workflow بال audit.
- بيئة يمكن أن تبني فوقها أدوات ذكاء اصطناعي متقدمة.
- نظامًا يمكن الوثوق به لأنه قابل للتتبع، والتحقق، والتدقيق.

هذه ليست مجرد خطة تطوير، بل خطة تحويل EngineeringOS إلى نواة معرفة هندسية قابلة للبناء فوقها.